

Difficulty calibration of the wildfire simulator

Re-choosing six configuration defaults so that an unmanaged `firefighter` team saves the home base in roughly one run in ten — with the simulation model and sweep methodology it was measured under.

Date	10 August 2026
Batch	experiments/20260810
Simulator	Wildfire-UAVSim, Mesa 1.2.1
Instrument	tools/sweep.py, headless.py
Runs	~20 400 simulated runs, ~30 min wall clock at 8 workers
Target	10% of runs won, unmanaged firefighter
Result	10.9% (95% Wilson 9.1 – 13.0), n = 1000 held-out seeds

Summary

The simulator's shipped configuration had never been calibrated against any notion of how hard a run is. An unmanaged `firefighter` team saved the base 37% of the time and `defend-base` 78% — too easy to leave a self-adaptive managing system measurable headroom, and too easy for the adaptation experiments the simulator exists to support.

Six defaults were re-chosen *together* — grid size, run length, fire spread speed, base health, and the two water-extinguish probabilities — from a 32-arm scan followed by a bisection on the one dial that responds smoothly rather than in a cliff. The result was confirmed on 1000 seeds that took no part in the fitting.

Several findings run against the obvious assumption and are recorded in full: a larger grid makes the scenario *easier*, stronger wind makes it easier, the fuel extension has no effect on outcomes at all, and smoke is not a simulation feature. One methodological defect was found and corrected mid-experiment; it is documented in §3.7 rather than quietly fixed.

Contents

1. Objective and outcome
2. The simulation model
3. Parameter sweep methodology
4. Results
5. The calibrated configuration
6. Appendix – full configuration and data index

1 Objective and outcome

Produce a configuration in which a basic, non-adaptive UAV policy wins about one run in ten: difficult, but not impossible. That band matters because the simulator's purpose is comparing adaptation strategies. Too easy and every strategy scores the same near the ceiling; too hard and every strategy scores zero. Around 10% there is room to distinguish them in both directions.

1.1 WHAT A RUN IS WON BY

There is exactly one outcome in this simulator, and it concerns the home base. A run is **won** by reaching the end of the step budget with the base still standing, and **lost** the moment the base has burned for `BHP` cumulative steps. Nothing else ends a run: not the fire going out, not the entire fleet being destroyed, not the map being consumed.

This has a consequence that shapes the whole experiment. A wildfire on a 100×100 map is never actually extinguished, so **given long enough, every run is lost**. Winning means outlasting the clock. The step budget is therefore part of the scenario definition rather than a free parameter, and it turns out to be the single strongest difficulty dial in the configuration (§4.2).

1.2 DECISIONS TAKEN BEFORE MEASURING

- **Calibrate against `firefighter`**. Of the two competent baseline policies it is the weaker; `defend-base` is reported alongside but not targeted.
- **With the managing system off** (`--managing none`). Calibrating the bare policy leaves the managing system measurable headroom, which is the point of the exercise. The managed arms are then a result rather than a target.
- **Keep uniform random ignition on a large grid** and recover the difficulty through the other dials, rather than coupling ignition placement to the base. This preserves the existing `FIRE_START_POSITION` semantics at the cost of some variance (§4.6).

1.3 OUTCOME

HEADLINE RESULT · SEED BLOCK 500000, USED FOR NOTHING ELSE · SHIPPED CONFIGURATION, NO OVERRIDES

POLICY	UNMANAGED	95% WILSON	HEURISTIC	95% WILSON	N
firefighter	10.9%	9.1 – 13.0	17.5%	15.3 – 20.0	1000
defend-base	19.6%	17.3 – 22.2	18.4%	16.1 – 20.9	1000
follow-fire	0.0%	0.0 – 0.8	—		500
random	0.0%	0.0 – 0.8	—		500

The scenario discriminates rather than merely being hard: two of the four policies never win, one wins about a tenth of the time, and the strongest wins about a fifth. The wins are also earned rather

than inherited — 100 of the 109 winning firefighter runs took base damage on the way, averaging 2.06 of the 4 points available, and losses are spread across steps 15 to 99 with a median of 50. A scenario in which winners never came under threat would indicate the fire simply never arrived.

2 The simulation model

What follows describes the model as configured for this experiment, in enough detail to interpret the results and to reproduce them. Every quantity named in capitals is a setting in `config.py`; the calibrated values are listed in §5 and the complete state in Appendix A.

2.1 GRID AND AGENTS

The world is a `WIDTH` × `HEIGHT` grid of cells stepped by Mesa's `SimultaneousActivation` scheduler: every agent computes its next state from a shared snapshot in `step()`, and all states are committed together in `advance()`. There is no activation ordering to randomise, which is one of the things that makes a run reproducible from a single seed.

Each cell independently holds a vegetation agent (`Fire`) with probability `DENSITY_PROB`. A cell without one can never burn, which is what makes density non-linear in its effect: below the percolation threshold the fire stalls, near 1.0 it sweeps the map. The ignition cell always receives an agent regardless of the draw.

Every vegetation cell starts with an integer fuel load drawn uniformly from `[ FUEL_BOTTOM_LIMIT, FUEL_UPPER_LIMIT ]` and loses `BURNING_RATE` per fire update while alight, so a cell burns for `fuel / BURNING_RATE` updates before burning out. This sets the *thickness* of the fire front, and hence how many burning neighbours a typical cell sees, without changing the per-step ignition probability directly.

2.2 FIRE SPREAD

A cell's probability of catching fire is one minus the probability that every burning neighbour fails to light it. The influence of a burning neighbour falls off with the inverse square of Euclidean distance and is cut off at a fixed radius:

$$P(s) = 1 - \prod_{s' \text{ burning}} (1 - w(s, s'))$$
$$w(s, s') = d(s, s')^{-2} \quad \text{for } d \leq 3, \text{ else } 0 \quad // \text{ Moore, radius } 3$$

The radius of 3 and the inverse-square law are **hard-coded** (`sim/agents/fire.py` and `sim/fire_spread.py`), not configurable, and they are the strongest untunable influence on spread rate. Note that an orthogonally adjacent burning cell has $d = 1$ and therefore $w = 1$: it ignites its neighbour with certainty, which is why the vectorised implementation carries a negative-infinity sentinel in its log-space kernel.

The live implementation evaluates this as a single convolution over the whole grid per step (`sim/fire_spread.py`), working in log space so the product becomes a sum. A readable per-cell version survives in `Fire.probability_of_fire()` and the two are pinned against each other by `tests/test_fire_spread.py`. The vectorisation is what makes a sweep of this size affordable at all — it took a full run from about 16 seconds to under one.

Each cell then draws once per fire update and ignites if the draw falls below its probability. Updates happen every `FIRE_SPREAD_SPEED` steps, so that setting is a pure time divider on the whole rule while UAVs, water drops and immunity countdowns continue to run every step. It is the cleanest available “give the team more or less time” dial, and at the calibrated value of 1 the fire updates every step.

2.3 WIND

Wind reshapes each pairwise influence before the product is taken, adding a share `MU` of the remaining probability downwind and removing the same share elsewhere:

```
downwind:      w ← w + MU · (1 - w)
otherwise:     w ← w - MU · w
```

This is not a conservative reshaping: for small w it raises the downwind term by more than it lowers the others, so strong wind does not merely redirect the fire, it concentrates it into a narrower and faster front. With `FIXED_WIND = False` the model composes two directions, `FIRST_DIR` with probability `FIRST_DIR_PROB` and `SECOND_DIR` otherwise, redrawn per cell per update at the offsets where the two directional kernels disagree. A perpendicular pair produces a diagonal front; the calibrated default is south/east at 0.8.

2.4 UAVS

`NUM_AGENTS` UAVs launch from the home base footprint. Each step a UAV receives an `Observation`, returns an `Action`, and the model applies it:

- **Observation** is a square of side $2 \cdot \text{UAV_OBSERVATION_RADIUS} + 1$ centred on the UAV — 81 cells at the default radius of 4 — reporting which cells are burning, where other UAVs are, and the UAV's own fuel and water. Cells outside the grid are simply absent, which is how the edges are represented.
- **Movement** is a direction plus a distance capped at `UAV_SPEED`. Note that speed (5) exceeds observation radius (4), so a UAV can fly past what it can see; the competent policies cap themselves at the observation radius to avoid this.
- **Collisions** occur when two or more UAVs end a step on the same cell, excluding the base footprint, which is shared airspace. Each UAV involved rolls for one whole health point with probability `UAV_COLLISION_DAMAGE_MEAN`; a UAV whose `UAV_HP` reaches zero is destroyed and takes no further part.

`SECURITY_DISTANCE` takes no part in any of this. It is the threshold used by the MR2 scoring heuristic and by the managing system's crowding test, not a physical separation.

2.5 FIREFIGHTING: THE BASE, WATER AND REIGNITION

The home base occupies a `BASE_SIZE` footprint anchored at `BASE_POSITION`, or at one quarter of the way into the grid when that is `None` — deliberately off-centre, and the ignition draw explicitly avoids it. The base accumulates one damage point per step in which *any* of its cells is alight, so a four-cell base burning entirely still accrues only one point per step. Damage is cumulative rather than

consecutive, because a burning cell goes out for a step when none of its neighbours are yet alight. At `BHP = 4` the base survives four such steps.

A UAV carries `UAV_WATER_CAPACITY` loads. Refilling is passive — a UAV standing on the base begins refilling by itself over `BASE_REFILL_STEPS` — but only `BASE_CAPACITY` UAVs can refill at once, so arrivals queue. At the shipped values (capacity 1, one load) this queue is a genuine bottleneck and one of the clearer constraints on a team's throughput.

A drop covers a disc of radius `WATER_DROP_RADIUS` and consumes the whole step, no movement. Each burning cell in the disc is extinguished with a probability falling linearly from the centre to the rim:

$$P_{\text{ext}}(\text{cell}) = \text{CENTRE} + (d / \text{RADIUS}) \cdot (\text{EDGE} - \text{CENTRE}), \quad d \leq \text{RADIUS}, \quad \text{else } 0$$

calibrated: `CENTRE = 0.76`, `EDGE = 0.48`, `RADIUS = 2`

An extinguished cell is immune for `REIGNITION_DELAY` steps, after which it burns normally again; it may also relight unaided each step with probability `SPONTANEOUS_REIGNITION_PROB`. Water therefore buys time rather than territory, which is why a run is eventually always lost.

2.6 FUEL

Each UAV carries `UAV_FUEL` units and is charged per step for the ground it covered, the water it was carrying while covering it, and simply for being airborne:

$$\text{cost} = (\text{idle} + \text{BURN_PER_CELL} \cdot \text{cells}^{\text{SPEED_EXPONENT}}) \cdot (1 + \text{WATER_PENALTY} \cdot \text{load})$$

`idle = 0` when parked on the base, so flying home to refuel is worth the trip
exponent 1.5: covering ground in one dash costs more than covering it slowly

A UAV that empties its tank loses all remaining health and is destroyed. Refuelling at the base takes `BASE_REFUEL_STEPS` and shares the slot with water. The same function is read by the policies when they estimate range, so the estimate and the charge cannot drift apart.

As §4.5 shows, none of this binds at the calibrated settings: a firefighter returns to base for water often enough that it refuels for free on the way.

2.7 POSITIONING ERROR

The extension corrupts each UAV's *measured* position while leaving ground truth — used for movement, collisions, fuel and scoring — untouched. What a policy sees, what teammates see, and what the managing system plans from are all affected. The offset is a fixed per-UAV bias drawn once at creation, plus per-step jitter, each a uniform integer per axis:

```

measured = clamp(true + bias + noise)    bias ~ U[-BIAS_MAX, BIAS_MAX] per axis,
drawn once                               noise ~ U[-NOISE_MAX, NOISE_MAX] per
axis, per step

```

The jitter is deliberately *not* drawn from the shared generator. It is a pure function of the UAV and the step number, so that asking a UAV where it thinks it is has no effect on the run — the web interface, a test, and the managing system can all read a fix at different points in the step without perturbing anything.

2.8 METRICS RECORDED

PER-RUN OUTPUTS, ALL RECORDED IN THE JSON OF EVERY RUN IN THIS BATCH

FIELD	MEANING
lost / outcome	whether the base was destroyed; the only win condition
base_burning_steps	damage accrued, out of <code>BHP</code>
steps_completed	where the run ended — the loss step, or the budget
mr1_total	cumulative fraction of each UAV's 81-cell window that was burning, summed over steps. Rewards fire being <i>watched</i> , not extinguished
mr2	count of UAV pairs closer than <code>SECURITY_DISTANCE</code> at the end of a step: accepted collision <i>risk</i> , not collisions
collisions / uavs_lost	cells found holding more than one UAV; UAVs destroyed
uavs_out_of_fuel / fuel_remaining	dry tanks, and what was left in each
water_drops / cells_extinguished / refills	drops made, cells actually put out, base services completed
burning_cells_final	grid state at the end of the run
fire_start_pos / fire_start_step	the resolved ignition, for post-hoc stratification

2.9 THE POLICIES MEASURED

- **firefighter** — greedy nearest-fire attack with a priority ladder: return to base on low fuel, return to base with no water, drop if fire is within the drop radius, otherwise fly towards the nearest unclaimed fire. Includes team-level target claiming and cell deconfliction, and caps its own speed at the observation radius. Its weakness is that it chases whatever is nearest *to itself*, so a team can disperse to the fire front and be nowhere near the base when the fire reaches it.
- **defend-base** — the same loop with a different target filter: only fire within `BASE_THREAT_RADIUS` of the base footprint, sorted by distance *to the base*, so the whole team converges on one front; otherwise it parks on the base, where the airspace is free and it refills and refuels passively.
- **follow-fire** — flies at fire with no team-level deconfliction at all.
- **random** — a floor.

3 Parameter sweep methodology

3.1 INSTRUMENT

`tools/sweep.py` was written for this experiment. It sits on top of the existing `headless.py` runner, which already provides parallel workers, per-run seed derivation, configuration validation, failure isolation and a JSON record of every run. What it adds is arm enumeration, the join that records *which* configuration produced each run, and the statistics. It uses the standard library only; the project ships three runtime dependencies and adding a fourth for a development tool was not warranted.

Two subcommands:

- `scan` — the cartesian product of repeated `--axis NAME=v1,v2,...`, over fixed `--base NAME=VALUE` settings, at `--runs` per arm. Writes one CSV row per run and prints a win-rate table.
- `calibrate` — bisection on one continuous knob towards a target win rate.

3.2 PROCESS ISOLATION

Each arm runs in its own `headless.py` subprocess rather than by calling the runner in-process. `apply_overrides()` and `seed_simulation()` mutate the global `config` module, so arms sharing a process leak settings into one another unless every arm happens to set every key any other arm sets — a failure that produces plausible numbers and no error. A subprocess costs about 0.2 s of interpreter startup, amortised across `--runs`, and removes the whole class of mistake.

3.3 SEEDING AND PAIRING

A run is fully determined by its seed: `--seed` sets both the module-level generator and `config.SYSTEM_RANDOM`, run i of a batch receives `seed + i`, and the numpy generator used by the vectorised spread is itself seeded from `SYSTEM_RANDOM`. This was verified empirically: identical invocations produce byte-identical metrics, sequentially and in parallel.

All arms of a scan share one seed block, so arm k 's run i sees the same fire in every arm and comparisons between arms are **paired**, which removes the ignition lottery from the contrast. The corresponding risk is over-fitting to one set of fires, addressed in §3.6.

A CAVEAT ON THE PARALLEL EXECUTOR

Seeding is process-global, so `--executor thread` with more than one worker destroys reproducibility. Every measurement in this batch used the default process executor.

3.4 DERIVED AXES

Some quantities worth sweeping are not a single constant. Water strength is the important one: a drop's effect falls linearly from `WATER_EXTINGUISH_PROB_CENTRE` to `_EDGE`, and sweeping the two independently would spend most of the arms on combinations nobody wants, including those where the rim is stronger than the centre. `EXTINGUISH_SCALE` scales both from the original 0.95 / 0.60,

preserving the shape of the falloff, so one number moves the whole water model and can be bisected on. `GRID` does the same for `WIDTH` and `HEIGHT`. Both are declared in a `DERIVED` table at the top of the tool.

3.5 SAMPLE SIZES AND INTERVALS

Every proportion is reported with a **Wilson score interval** rather than the normal approximation. Near a 10% rate the normal interval is badly calibrated and its lower bound can fall below zero; Wilson stays within $[0, 1]$ and behaves sensibly at zero successes, which matters here because several arms won nothing at all and “0/400” must be distinguishable from “unwinnable”.

SAMPLE SIZES USED, AND WHAT EACH RESOLVES AT A 10% RATE

STAGE	N PER ARM	95% INTERVAL HALF-WIDTH	PURPOSE
coarse scan	100	$\pm \sim 6$ pp	locate the crossing, rank arms
calibration	400	$\pm \sim 3$ pp	choose the value
characterisation	200	$\pm \sim 4$ pp	direction and rough size of each dial
validation	1000	$\pm \sim 1.9$ pp	the number that gets reported

Cost was not a constraint: at roughly 0.35 s per run, the entire design is about one core-hour. Sample sizes were chosen for interval width, not budget.

3.6 SEARCH STRATEGY

1. **Fix the structural settings.** Several knobs are cliffs rather than slopes — one integer step of `UAV_POSITION_NOISE_MAX`, `WATER_DROP_RADIUS`, `REIGNITION_DELAY` or `BASE_REFILL_STEPS` takes the scenario from difficult to unwinnable. None can be used for fine adjustment, so each was settled as a structural choice and held fixed.
2. **Coarse scan** over `FIRE_SPREAD_SPEED` $\times$ `BHP` $\times$ `EXTINGUISH_SCALE`, 32 arms at $n = 100$.
3. **Select by gradient, not by proximity.** The chosen arm is not the one nearest 10% but the one whose win rate responds *least sharply* to the fine dial, so the configuration is robust to a small mis-set. This was `FIRE_SPREAD_SPEED = 1`, `BHP = 4`, at roughly half the local slope of `BHP = 5`, and crossing the target in the interior of the range rather than at an end.
4. **Bisect** the fine dial at $n = 400$, stopping when the target lies inside a probe's interval — past that point halving again only reads noise. The direction is measured from the two endpoints rather than assumed, because the knobs in this configuration do not agree on one (a larger grid and a larger `BHP` both make the scenario *easier*).
5. **Refine** with a small scan either side of the crossing, and choose a value that also yields round two-decimal constants.
6. **Validate on a disjoint seed block** at $n = 1000$. Everything above shares seed block 1000, so the tuned value is fitted to one set of fires; the reported figure comes from block 500000, used for nothing else. This is the main way an experiment of this shape fails quietly, so it is not optional.
7. **Characterise** the remaining dials one factor at a time from the tuned point, so the record documents the surface rather than only the chosen spot.

“--STEPS” DID NOT SET THE RUN LENGTH

The first pass used `headless.py --steps` to control how long a run lasted. At the time it did not do that. It was a separate count of how many times the runner called `step()`, while the model stops itself once its own `config.BATCH_SIZE` is reached — two numbers for one quantity, wrong in both directions:

- **Above `BATCH_SIZE` the flag did nothing.** The model had already stopped. A sweep over 120, 150 and 200 steps returned three *identical* win rates, which read convincingly as evidence that **run length does not matter**. The opposite is true — it is the strongest dial in the configuration (§4.2), and every measurement had in fact been taken at an effective budget of 100 steps.
- **Below `BATCH_SIZE` the loop stopped early** — and a run cut short has not lost its base yet, so it was scored **WON**. This half would have inflated any arm that shortened a run, silently and in the flattering direction.

The defect surfaced only at final verification, when the tuned `config.py` produced 3.5% where the sweeps had predicted 10.9%. No figure in this report was affected by the second half — every arm used a `--steps` at or above `BATCH_SIZE`, so no run was truncated — and because all the sweeps shared the same erroneous budget, they remained internally consistent and were re-usable once `BATCH_SIZE` was set to 100 to match them. Only the run-length conclusion had to be discarded and re-measured (§4.2); that sweep was then re-run through `--set BATCH_SIZE` and is the data in `batch-size/`.

Corrective action: `headless.py --steps N` is now an alias for `--set BATCH_SIZE=N`, folded into the overrides so there is genuinely one number; passing both with different values is an error rather than a silent winner. `tests/cli/test_run_length.py` covers both directions end to end. `tools/sweep.py` declines `--steps` regardless, since one axis should have one name, and `config.py` and the README now document the constant as the only run length there is.

3.8 OTHER THREATS TO VALIDITY

- **Ignition variance dominates within an arm.** Under uniform random ignition, whether the fire starts near the base explains much of the run-to-run variation. Paired seeds remove this from *between-arm* comparisons but not from the absolute rate, which is why $n = 1000$ was used for the reported figure.
- **Single-policy calibration.** The target was set on `firefighter` alone. `defend-base` lands at 19.6% and was not tuned towards; a configuration making both simultaneously 10% may not exist without a dial that penalises them differently.
- **One-factor-at-a-time characterisation.** §4.5 varies dials individually from the tuned point and therefore cannot detect interactions away from it. At least one interaction is known to exist and to be non-monotone (observation radius, §4.6).
- **Vectorised-spread seeds are not comparable to archived pre-vectorisation results.** Self-consistent within this batch.

4 Results

4.1 BASELINE, BEFORE ANY CHANGE

ORIGINAL SHIPPED DEFAULTS · 50×50 · N = 60 PER CELL · UNMANAGED

CHANGE FROM DEFAULTS	FIREFIGHTER	DEFEND-BASE
none (baseline)	37%	78%
grid 100×100	65%	87%
grid 150×150	80%	88%
FIRE_SPREAD_SPEED = 1	20%	40%
BHP = 3	23%	52%
NUM_AGENTS = 2	30%	62%
UAV_FUEL = 50	32%	73%
MU = 0.8	50%	87%
UAV_OBSERVATION_RADIUS = 2	57%	90%
extinguish 0.70 / 0.30	8%	33%
extinguish 0.50 / 0.20	2%	3%
WATER_DROP_RADIUS = 1	2%	7%
REIGNITION_DELAY = 2	2%	7%
UAV_POSITION_NOISE_MAX = 2	0%	0%
UAV_POSITION_BIAS_MAX = 2	7%	5%

Two things are visible immediately. The two baseline policies are 41 points apart, so a single configuration cannot put both at 10%. And most single knobs are cliffs: between the two extinguish rows the scenario passes the target and lands at 2%, and one step of positioning noise takes it to zero.

4.2 RUN LENGTH

WIN RATE AGAINST THE STEP BUDGET · TUNED SETTINGS OTHERWISE · N = 400 · FIREFIGHTER, UNMANAGED

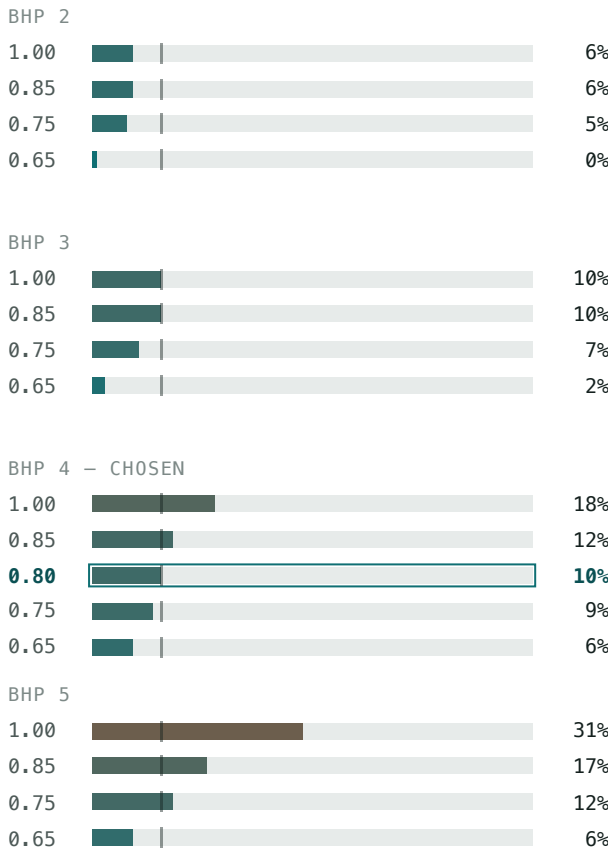
BATCH_SIZE	WON	RATE	95% WILSON	LATEST LOSS	P90 LOSS
100	40/400	10.0%	7.4 – 13.3	99	79
125	17/400	4.2%	2.7 – 6.7	125	89
150	10/400	2.5%	1.4 – 4.5	150	92
200	2/400	0.5%	0.1 – 1.8	194	99
300	0/400	0.0%	0.0 – 1.0	214	101
400	0/400	0.0%	0.0 – 1.0	214	101

The p90 loss step saturates around 101 and the latest observed loss at 214, confirming §1.1: given enough time the fire always reaches and destroys the base. `BATCH_SIZE` = 100 is therefore a calibrated value, not an inherited one.

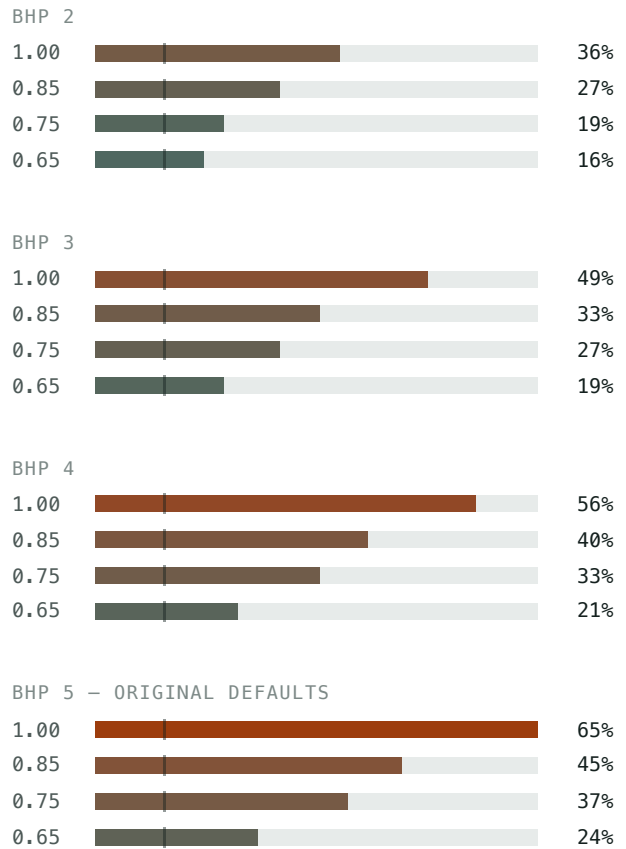
4.3 THE COARSE SURFACE

32 ARMS · 100×100, 100 STEPS · N = 100 PER ARM · SEED BLOCK 1000 · FIREFIGHTER, UNMANAGED

FIRE_SPREAD_SPEED = 1 — every step



FIRE_SPREAD_SPEED = 2 — the original



Left column is `EXTINGUISH_SCALE`; bars run 0–65% of runs won; the hairline marks the 10% target. The outlined 0.80 row is the $n = 400$ refinement, not part of the 32-arm scan.

At `FIRE_SPREAD_SPEED = 2` no arm reaches the target — the lowest is 16% — so the original spread speed cannot produce this scenario on a 100×100 grid at all. Within spread speed 1, `BHP = 2` is already below target at full water strength, `BHP = 3` crosses at the very edge of the range, and `BHP = 5` crosses steeply. `BHP = 4` is the only arm that crosses gently in the interior.

4.4 CALIBRATION

BISECTION AND REFINEMENT ON THE WATER SCALE · FIRE_SPREAD_SPEED = 1, BHP = 4 · N = 400 · SEED BLOCK 1000

EXTINGUISH_SCALE	CENTRE / EDGE	WON	RATE	95% WILSON
0.65	0.62 / 0.39	17/400	4.2%	2.7 – 6.7
0.74	0.70 / 0.44	21/400	5.2%	3.5 – 7.9
0.78	0.74 / 0.47	35/400	8.8%	6.4 – 11.9
0.80	0.76 / 0.48	40/400	10.0%	7.4 – 13.3
0.82	0.78 / 0.49	39/400	9.8%	7.2 – 13.1
0.825	0.78 / 0.50	47/400	11.8%	9.0 – 15.3
1.00	0.95 / 0.60	84/400	21.0%	17.3 – 25.3

The curve is flat between 0.78 and 0.82, which is the property worth having: the scenario does not hinge on the third decimal place of a probability. The 0.82 and 0.825 rows differ by 2 points with heavily overlapping intervals, which is a fair picture of the noise floor at this sample size.

4.5 CHARACTERISATION OF THE REMAINING DIALS

One factor at a time from the tuned configuration, n = 200 per arm, firefighter unmanaged, seed block 1000. The tuned point itself reads 10.0% in this series.

FUEL – AND WHETHER IT BINDS AT ALL

UAV_FUEL	RATE	RAN DRY / RUN	REFILLS / RUN	DRIEST TANK LEFT
40	9.5%	0.29	4.0	9.8
60	9.5%	0.04	4.2	29.7
100	10.0%	0.01	4.2	69.6
150	10.0%	0.00	4.2	119.9
250	10.0%	0.00	4.2	219.9

At the shipped tank a UAV uses about 30 of its 150 units across four refills, and no UAV ever runs dry. The fuel model is entirely masked by the water logistics: a firefighter returns to base for water often enough that it refuels for free on the way. UAV_FUEL_WATER_PENALTY over {0.0, 0.3, 0.6, 1.0} produced **20/200 in every case** — literally no effect. Making fuel a live constraint would require decoupling refuelling from refilling, not a different value.

SETTING	RATE	95% WILSON	SETTING	RATE	95% WILSON
MU = 0.0	9.0%	5.8 – 13.8	GRID 50	5.0%	2.7 – 9.0
MU = 0.25	6.0%	3.5 – 10.2	GRID 75	11.0%	7.4 – 16.1
MU = 0.5	10.0%	6.6 – 14.9	GRID 100	10.0%	6.6 – 14.9
MU = 0.8	21.0%	15.9 – 27.2	GRID 150	12.5%	8.6 – 17.8
WATER_CAP 3, BASE_CAP 1	7.0%	4.2 – 11.4	noise 0, bias 0	21.0%	15.9 – 27.2
WATER_CAP 3, BASE_CAP 2	8.0%	5.0 – 12.6	noise 0, bias 1	13.0%	9.0 – 18.4
WATER_CAP 1, BASE_CAP 1	10.0%	6.6 – 14.9	noise 1, bias 0	10.0%	6.6 – 14.9
WATER_CAP 1, BASE_CAP 2	12.5%	8.6 – 17.8	noise 1, bias 1	7.0%	4.2 – 11.4

4.6 FINDINGS THAT RUN AGAINST THE ASSUMPTION

A bigger grid makes the scenario easier

`FIRE_START_POSITION` draws uniformly over the map, so a larger grid usually ignites further from the base and the fire often never arrives. At the original spread speed a firefighter team won 37% at 50×50 and 80% at 150×150 — and 83% at 150×150 even when given four times the budget. Raising the spread speed is what decoupled the two: the tuned configuration reads 5% / 10% / 12.5% at 50 / 100 / 150, i.e. grid size is now close to a free choice. This was the tension behind the decision recorded in §1.2, and it resolved favourably.

Stronger wind makes the scenario easier, non-monotonically

`MU = 0.8` doubles the win rate. A strong wind concentrates the fire into a directed ray that usually misses a small base, so it is a *shape* parameter rather than an intensity one. The response is not even monotone — 9.0% at 0, 6.0% at 0.25, 10.0% at 0.5, 21.0% at 0.8 — so it is unsuitable as a difficulty dial in either direction.

Smoke is not a simulation feature

`Smoke` is read by `sim/gui/portrayal.py` and by nothing else. It does not obscure a UAV's observation, does not reach the managing system, and takes no part in how the fire spreads. `ACTIVATE_SMOKE` changes what the canvas looks like and nothing about a run, so “smoke / no smoke” could not be an experimental axis. Making smoke occlude observation would be a change to `UAV.observe()` and `Observation` — a feature to build, not a setting to vary.

Reducing what a UAV can see can improve its team

Cutting `UAV_OBSERVATION_RADIUS` from 4 to 2 raised `defend-base` from 78% to 90%. A UAV that sees no fire parks on the base — which is where it needed to be. Vision interacts with policy structure rather than acting as a straightforward difficulty dial, and this is the known interaction that limits the one-factor-at-a-time reading of §4.5.

Positioning error accounts for about half the difficulty

With the error off entirely the same team wins 21% instead of 10.9%. Roughly half the difficulty of the calibrated scenario is therefore sensor error rather than fire, which is worth stating whenever a

result is attributed to the fire model. It is also effectively a binary dial: `NOISE_MAX = 2` lost all 60 runs for both competent policies, and a fixed bias of 2 was nearly as absolute.

4.7 A RESULT ABOUT THE MANAGING SYSTEM, NOT THE CONFIGURATION

The heuristic managing system is worth **+6.6 points to a firefighter team** (10.9% → 17.5%, non-overlapping intervals) and **nothing at all to a defend-base team** (19.6% → 18.4%, intervals overlapping). Taken with an earlier finding that it *increases* collisions under the shipped positioning error — 47 against an unmanaged baseline's 18, where with perfect information it scores 0 against 40 — that is three independent measurements in which it helps one policy, does nothing for another, and harms a third. This is recorded here as an observation arising from the calibration; investigating it is separate work.

5 The calibrated configuration

THE SIX SETTINGS CHOSEN TOGETHER

SETTING	WAS	NOW	WHY THIS VALUE
WIDTH, HEIGHT	50, 50	100, 100	largest grid the 10 px/cell canvas accommodates; difficulty is now roughly independent of size
BATCH_SIZE	100	100	unchanged, but now a calibrated value: 125 gives 4.2%, 150 gives 2.5%
FIRE_SPREAD_SPEED	2	1	at 2 the fire frequently never reaches the base on a 100×100 map; no arm reached the target
BHP	5	4	the value at which the win rate responds most gently to the water strength
WATER_EXTINGUISH_PROB_CENTRE	0.95	0.76	the fine dial; <code>EXTINGUISH_SCALE</code> = 0.80, chosen by bisection then refinement
WATER_EXTINGUISH_PROB_EDGE	0.60	0.48	

5.1 SETTINGS DELIBERATELY LEFT ALONE

- **Positioning error** stays at `NOISE_MAX = 1`, `BIAS_MAX = 0`. One step further is unwinnable for every policy, so 1 is the only playable non-zero value.
- **Water drop radius, reignition delay, refill steps, base capacity** — each a cliff; one step of any of them overshoots the target entirely.
- **Wind** — weak, shape-changing and non-monotone (§4.6).
- **Fuel** — has no effect on outcomes at any value tested (§4.5).
- **Smoke** — left on because it costs nothing and looks right; it is not a simulation feature.

5.2 REPRODUCING THIS BATCH

```
# the 32-arm coarse surface (§4.3)
python3 tools/sweep.py scan --policy firefighter --managing none --runs 100 --seed 1000 \
  --base WIDTH=100 --base HEIGHT=100 --base BATCH_SIZE=100 \
  --axis FIRE_SPREAD_SPEED=1,2 --axis BHP=2,3,4,5 \
  --axis EXTINGUISH_SCALE=1.0,0.85,0.75,0.65 --out experiments/20260810/coarse

# the fine dial, bisected to a target (§4.4)
python3 tools/sweep.py calibrate --policy firefighter --managing none --knob
EXTINGUISH_SCALE \
  --low 0.65 --high 1.0 --target 0.10 --runs 400 --seed 1000 \
  --base WIDTH=100 --base HEIGHT=100 --base FIRE_SPREAD_SPEED=1 --base BHP=4 \
  --out experiments/20260810/calibrate

# run length (§4.2); --steps N is now an alias for --set BATCH_SIZE=N (§3.7)
python3 tools/sweep.py scan --policy firefighter --managing none --runs 400 --seed 1000 \
  --axis BATCH_SIZE=100,125,150,200,300,400 --out experiments/20260810/batch-size

# the reported figure: held-out seeds, shipped config, no overrides (§1.3)
python3 headless.py --runs 1000 --workers 8 --seed 500000 \
  --policy firefighter --managing none --log-every 0 \
  --output experiments/20260810/validate/firefighter__none.json
```

Appendix A Full configuration

The complete simulation state under which every figure in §4.3 onwards was produced, except where a table names an override. Calibrated values are marked.

SETTING	VALUE	SETTING	VALUE
WIDTH, HEIGHT	100, 100 ★	ACTIVATE_FIREFIGHTING	True
BATCH_SIZE	100 ★	BASE_POSITION	None
DENSITY_PROB	0.9	BASE_SIZE	(2, 2)
FUEL_BOTTOM_LIMIT	5	BHP	4 ★
FUEL_UPPER_LIMIT	10	BASE_CAPACITY	1
BURNING_RATE	1	BASE_REFILL_STEPS	1
FIRE_SPREAD_SPEED	1 ★	UAV_WATER_CAPACITY	1
FIRE_START_POSITION	'random'	WATER_DROP_RADIUS	2
FIRE_START_STEP	(10, 20)	WATER_EXT_PROB_CENTRE	0.76 ★
ACTIVATE_WIND	True	WATER_EXT_PROB_EDGE	0.48 ★
FIXED_WIND	False	REIGNITION_DELAY	8
FIRST_DIR / SECOND_DIR	south / east	SPONTANEOUS_REIGNITION_PROB	0.005
FIRST_DIR_PROB	0.8	NUM_OUT_BUILDINGS	0
MU	0.5	ACTIVATE_FUEL	True
NUM_AGENTS	4	UAV_FUEL	150
UAV_SPEED	5	UAV_FUEL_IDLE_BURN	1.0
UAV_OBSERVATION_RADIUS	4	UAV_FUEL_BURN_PER_CELL	1.0
N_OBSERVATIONS	81	UAV_FUEL_SPEED_EXPONENT	1.5
UAV_HP	3	UAV_FUEL_WATER_PENALTY	0.3
UAV_COLLISION_DAMAGE_MEAN	1.0	UAV_FUEL_RESERVE	0.25
SECURITY_DISTANCE	2	BASE_REFUEL_STEPS	2
ACTIVATE_SMOKE	True (cosmetic)	ACTIVATE_POSITION_ERROR	True
MANAGING_SYSTEM	'heuristic'	UAV_POSITION_BIAS_MAX	0
DEFAULT_UAV_POLICY	'firefighter'	UAV_POSITION_NOISE_MAX	1
ADAPTATION_PERIOD	1	BASE_SENSOR_RADIUS	6
ADAPTATION_HYSTERESIS	2	BASE_THREAT_RADIUS	10

★ calibrated in this batch. Fire neighbourhood radius (3) and the inverse-square falloff are hard-coded in `sim/agents/fire.py` and `sim/fire_spread.py` and cannot be set from `config.py`. Every sweep arm ran with `MANAGING_SYSTEM` overridden by `--managing none` except where a table says otherwise.

Appendix B Data index

Everything below is under `experiments/20260810/`. Each directory holds one JSON file per arm (the raw `headless.py` output for every run) and, for sweeps, a `runs.csv` in long format with the swept parameters joined onto each run.

DIRECTORY	CONTENTS	RUNS	SECTION
<code>coarse/</code>	32-arm surface, spread $\times$ BHP $\times$ water	3 200	§4.3
<code>calibrate/</code>	bisection probes on the water scale	1 200	§4.4
<code>fine/</code>	refinement either side of the crossing	1 600	§4.4
<code>batch-size/</code>	win rate against the step budget	2 400	§4.2
<code>validate/</code>	held-out seeds, four policies, managed and not	7 000	§1.3
<code>char-fuel/</code>	UAV_FUEL	1 000	§4.5
<code>char-penalty/</code>	UAV_FUEL_WATER_PENALTY	800	§4.5
<code>char-wind/</code>	MU	800	§4.5
<code>char-grid/</code>	GRID	800	§4.5
<code>char-water/</code>	UAV_WATER_CAPACITY $\times$ BASE_CAPACITY	1 200	§4.5
<code>char-position/</code>	NOISE_MAX $\times$ BIAS_MAX	800	§4.5

The §4.1 baseline scan was run interactively and its raw output was not retained; the table reproduces the logged win rates. Every other figure in this report is recoverable from the files above.